IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

IA Racing

TAF: ASCy - ROBIN

Procom

Élèves :

Anthony CENZANO

Zaineb MAHMOUDI

Wissal DAHANI

Lixi WANG

Chen ZHANG

Enseignants :

Fabien CLAVEAU

Yuqi LIU

Alessandro COLOTTI

29 mars 2026

IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

Table des matières

1 Introduction et Contexte **3**

2 Description du Système **4**

2.1 Architecture matérielle 4

2.2 Architecture logicielle 4

3 Méthodologie **5**

3.1 Perception 5

3.1.1 Transformation BEV (*Bird's Eye View*) 6

3.1.2 Filtrage des couleurs en HSV 6

3.1.3 ROI (*Region of Interest Masking*) 7

3.1.4 Filtrage des zones de voie 8

3.1.5 Nettoyage doux et fusion des segments 9

3.1.6 Squelettisation et clustering (*Skeletonization & DBSCAN*) 9

3.1.7 Ajustement de courbe et visualisation 10

3.1.8 Calcul des coordonnées de la ligne de voie, de l'angle de déviation du véhicule et du décalage latéral 11

3.1.9 Principales difficultés du module de perception 12

3.2 Commande 13

3.2.1 Principe du contrôleur Stanley 14

3.2.2 Architecture générale 14

3.2.3 Fonctionnement du code 15

3.2.4 Paramètres principaux 15

3.2.5 Points d'attention 16

3.2.6 Simulation 16

3.3 Calibration et mesures 17

4 Résultats et Discussion **18**

5 Conclusion **19**

6 Bibliographie **21**

1 Introduction et Contexte

Ce projet vise à réaliser un DonkeyCar capable de suivre une voie de manière autonome à l'aide d'une caméra embarquée et d'un contrôleur classique de type Stanley [1], sans apprentissage profond. Le travail a été organisé en trois sous-systèmes principaux : la perception (détection de la voie), la commande (calcul de l'angle de braquage et de l'accélération) et la calibration/mesure (caméra, odométrie et outils d'analyse hors ligne). La structuration perception–commande retenue s'inscrit dans la continuité des approches classiques de détection et de suivi de voie décrites dans la littérature [2, 3]. Le véhicule repose sur une plateforme Raspberry Pi avec la bibliothèque DonkeyCar, pilotant un châssis de voiture radiocommandée équipé d'un ESC et d'un servo de direction.

2 Description du Système

2.1 Architecture matérielle

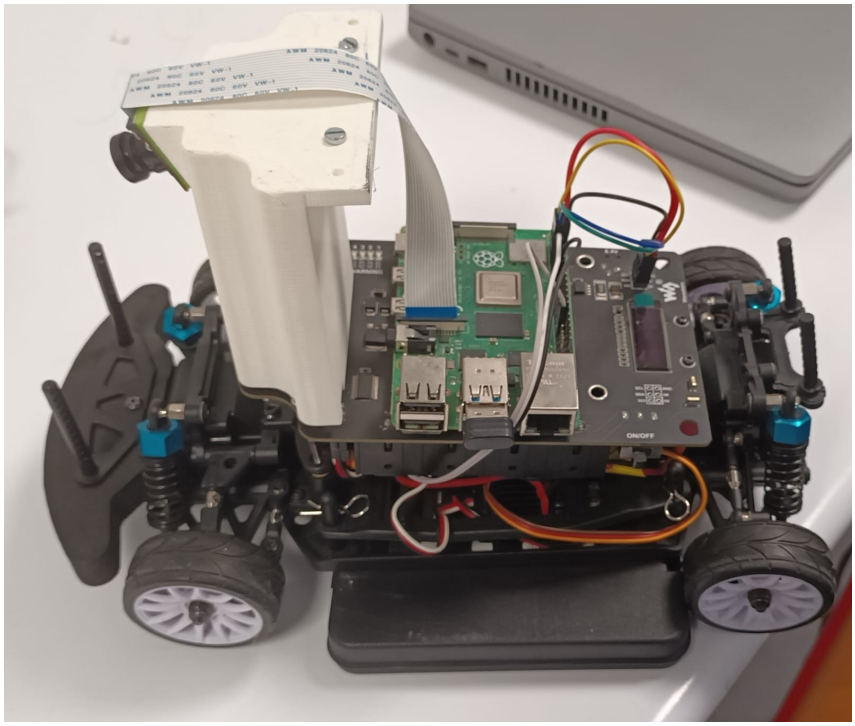


FIGURE 1 – DonkeyCar équipé : Raspberry Pi, caméra PiCamera, PCA9685, ESC et servo de direction.

Le robot est constitué d'un châssis de voiture RC, d'un Raspberry Pi, d'un contrôleur PWM (type PCA9685) pour piloter le servo de direction et l'ESC, et d'une caméra montée à l'avant pour observer la piste. L'alimentation est assurée par la batterie du châssis, avec une régulation de tension pour le Raspberry Pi.

2.2 Architecture logicielle

Sur le plan logiciel, le projet s'appuie sur l'architecture en *parts* de DonkeyCar, où chaque module lit et écrit des variables dans la mémoire du véhicule à chaque itération de la boucle. Les éléments principaux développés sont : `LaneDetectorPart` pour calculer l'offset latéral et l'angle de la voie à partir de l'image caméra, `OdometryPart` pour estimer la vitesse

et le yaw, `StanleyControllerPart` pour générer les consignes de braquage et de throttle, `AutoPilotPart` pour fournir `pilot/angle` et `pilot/throttle`, et `DataLoggerPart` pour enregistrer les signaux dans des fichiers CSV. L'ensemble de ces *parts* est intégré dans la fonction `drive` de `manage.py`, qui orchestre la chaîne de traitement en temps réel.

Le code source est disponible sur GitHub :
`https://github.com/ant965/DonkeycarStanleyAutopilot`

3 Méthodologie

3.1 Perception

L'objectif de cette expérience est d'extraire de manière stable et précise les lignes de voie gauche et droite à partir des images de la caméra frontale du véhicule, puis d'obtenir une courbe ajustée et lissée. L'ensemble du traitement reprend les grandes étapes des pipelines classiques de détection de voie fondés sur la transformation de perspective, la segmentation et l'ajustement géométrique [2, 3, 4]. Il se compose de huit étapes principales :

1. transformation en perspective en vue zénithale (*Bird's Eye View*, BEV) ;
2. filtrage des couleurs en HSV ;
3. extraction de la région d'intérêt (ROI) ;
4. filtrage des zones de voie ;
5. nettoyage doux et fusion des segments ;
6. squelettisation et clustering ;
7. ajustement de courbe et visualisation ;
8. calcul des points de voie et des paramètres géométriques.

La sortie de chaque étape sert d'entrée à la suivante, ce qui forme une chaîne complète de traitement d'images. Cette approche modulaire suit les recommandations générales de la littérature pour concilier robustesse et exécution temps réel [2, 4].

3.1.1 Transformation BEV (*Bird's Eye View*)

Dans l'image brute capturée par la caméra, la position frontale et inclinée du capteur provoque une forte distorsion en perspective : les lignes de voie apparaissent larges au premier plan, étroites au loin, et peuvent même sembler converger dans les virages.

Pour obtenir une structure géométrique plus facile à analyser, il est nécessaire de projeter l'image dans une vue en plongée. Ce procédé repose sur une matrice d'homographie calculée à partir de points correspondants entre le plan image et le plan du sol. La matrice de transformation M permet ainsi de projeter l'image du point de vue de la caméra vers un système de coordonnées en vue de dessus, ce qui assure une correction géométrique conforme aux approches de transformation de perspective utilisées pour le lane keeping [3].

Dans les résultats obtenus, l'image BEV présente une structure de voie beaucoup plus régulière. En particulier dans les virages, les lignes deviennent plus parallèles et plus continues, ce qui améliore fortement la qualité de la perception visuelle.

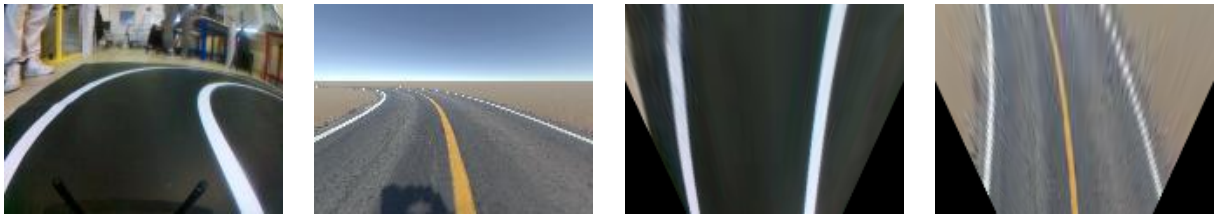


FIGURE 2 – Exemples de transformation en vue zénithale sur plusieurs cas de piste.

3.1.2 Filtrage des couleurs en HSV

L'environnement routier est complexe et comporte des ombres, des réflexions ainsi que des marques au sol dégradées. Pour extraire correctement les lignes de voie dans la vue BEV, nous utilisons une méthode de segmentation par couleur basée sur l'espace HSV, dans la continuité des approches de segmentation colorimétrique dédiées à la détection de voie [5].

Après la conversion de l'image du format BGR vers l'espace HSV, il devient possible d'analyser séparément la teinte (*Hue*), la saturation (*Saturation*) et la luminosité (*Value*), ce qui permet une sélection plus stable qu'un seuillage classique en niveaux de gris. Dans

notre expérimentation, nous conservons principalement les lignes blanches tout en excluant les lignes jaunes et les zones de forte réflexion lumineuse, suivant le principe selon lequel les composantes chromatiques séparent mieux les marquages routiers du fond [5, 2].

- Les zones blanches correspondent à une faible saturation et à une forte luminosité.
- Les zones jaunes occupent une plage de teinte spécifique, avec une saturation plus marquée.

Cette première segmentation basée sur la couleur réduit fortement les interférences du fond, et l'image binaire obtenue contient principalement les marques au sol de la route.

FIGURE 3 – Résultat après filtrage des couleurs dans l'espace HSV.

3.1.3 ROI (*Region of Interest Masking*)

Après l'obtention de l'image binaire, il reste encore de nombreuses zones sans intérêt pour la détection, comme le ciel, les bords de la piste, ou encore les éléments environnants. Afin de nous concentrer uniquement sur la partie utile de la route située devant le véhicule, nous avons conçu un masque ROI en forme de pentagone, qui conserve principalement la zone centrale et inférieure de la piste, comme cela est couramment recommandé dans les chaînes de détection de voie pour limiter les faux positifs [2, 4].

Les sommets du ROI sont calculés dynamiquement en fonction de la taille et des proportions de l'image, ce qui produit une région étroite en haut et large en bas. En appliquant ce masque à l'image binaire via une opération `AND`, nous éliminons efficacement les zones inutiles situées en haut et sur les côtés, tout en conservant la zone de roulement

utile [3, 4].

Les effets recherchés sont les suivants :

- conserver uniquement la zone liée à la trajectoire de conduite ;
- réduire la charge de calcul dans les étapes suivantes ;
- améliorer la précision de l'extraction des lignes de voie.

FIGURE 4 – Application du masque ROI sur la zone utile de la route.

3.1.4 Filtrage des zones de voie

Même à l'intérieur du ROI, la route peut contenir des textures complexes, ce qui génère encore des points ne correspondant pas aux lignes de voie. Pour affiner davantage la zone utile, cette étape analyse la distribution de luminosité dans la direction horizontale et détermine dynamiquement la position centrale des voies à partir des pics de distribution, dans l'esprit des méthodes de localisation progressive des marquages routiers [4].

L'algorithme génère ensuite un masque à largeur dynamique, plus étroit en bas de l'image et progressivement plus large en haut. Cette conception reproduit l'effet de perspective, ce qui permet de conserver les petits segments éloignés tout en réduisant les fausses détections. Le résultat final ne conserve que les zones situées autour des deux lignes principales, ce qui augmente la fiabilité de l'analyse des composantes connexes dans les étapes suivantes [3, 4].

L'objectif essentiel de cette étape est d'identifier automatiquement, par une méthode statistique, la zone latérale approximative des voies sans dépendre d'un réglage manuel des bords gauche et droit.

3.1.5 Nettoyage doux et fusion des segments

Après les étapes précédentes, l'image peut encore contenir de petites zones de bruit, des segments discontinus ou des blocs anormaux. Pour obtenir des lignes de voie continues et lisses, cette étape comprend deux parties complémentaires, cohérentes avec les étapes de raffinement morphologique souvent employées en détection de voie [2, 4] :

1. **Nettoyage doux.** Une analyse des composantes connexes mesure, pour chaque région de pixels, sa surface, son rapport largeur/hauteur ainsi que sa position centrale. Les petites zones de bruit et les régions trop larges horizontalement sont automatiquement supprimées, tandis que les segments verticaux plausibles sont conservés.
2. **Fusion des segments.** Les moitiés gauche et droite de l'image subissent séparément une fermeture morphologique afin de relier les segments discontinus appartenant à une même voie.

Après cette étape, l'image obtenue ne contient plus que les deux lignes de voie, gauche et droite, qui sont propres et continues. Les bruits et fragments résiduels sont fortement réduits, ce qui facilite les traitements géométriques ultérieurs [4].

FIGURE 5 – Nettoyage doux puis fusion morphologique des segments de voie.

3.1.6 Squelettisation et clustering (*Skeletonization & DBSCAN*)

Après le nettoyage de la zone de voie, l'objectif suivant est de réduire la largeur des lignes afin d'obtenir une ligne médiane en un seul pixel, ce qui diminue la quantité de données à traiter. Pour cela, nous utilisons la détection de contours de Canny [6] combinée à une opération morphologique de fermeture, puis un algorithme de squelettisation (*thinning*)

pour extraire l'axe central de la ligne [7].

L'image après squelettisation ne conserve que des lignes d'un pixel de largeur, ce qui réduit considérablement les données redondantes tout en préservant la géométrie des lignes de voie. Les points du squelette sont ensuite regroupés à l'aide de l'algorithme DBSCAN (*Density-Based Spatial Clustering of Applications with Noise*) [8].

Contrairement à *K-means*, DBSCAN ne nécessite pas de spécifier le nombre de clusters à l'avance et peut automatiquement distinguer les points de bruit des zones de haute densité [8].

- Les points de bruit (`label = -1`) sont supprimés automatiquement.
- Les clusters trop petits sont filtrés.
- Lorsque plusieurs clusters valides sont détectés, le coefficient de silhouette peut être calculé pour évaluer la qualité du clustering.

Les résultats expérimentaux montrent que ce processus parvient à séparer correctement les deux grands clusters correspondant aux lignes de voie gauche et droite, avec un bon niveau d'automatisation et une forte résistance au bruit, en accord avec l'intérêt de ce type de regroupement pour les données bruitées [8].

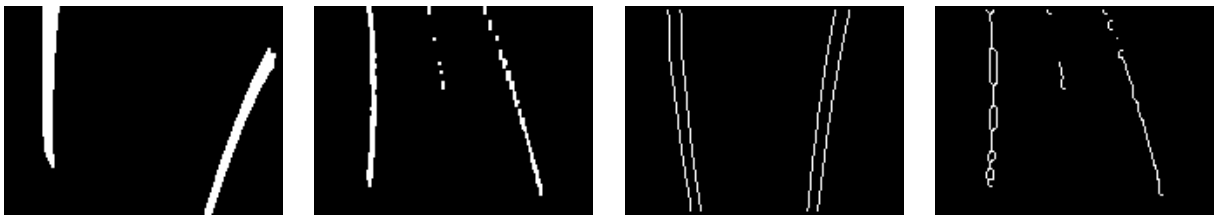


FIGURE 6 – Squelettisation puis clustering des points de voie avant ajustement.

3.1.7 Ajustement de courbe et visualisation

Après l'étape de clustering, la distribution des points de chaque cluster de voie reste discrète. Pour obtenir un modèle continu et exploitable pour l'analyse, nous utilisons un modèle polynomial du second degré, conforme aux approches d'ajustement de courbe utilisées en détection robuste de voie [4] :

$$x = ay^2 + by + c$$

Chaque ligne de voie est ajustée à l'aide de la méthode des moindres carrés. Dans l'image BEV, les valeurs de y sont naturellement ordonnées alors que les valeurs de x peuvent varier selon la courbure. Utiliser la forme $x = f(y)$ garantit donc une meilleure stabilité d'ajustement que la forme $y = f(x)$, en particulier dans les virages ou lorsque la ligne devient presque verticale [4].

Lorsqu'un seul cluster est détecté, une seule ligne de voie est tracée ; lorsqu'au moins deux clusters sont détectés, les clusters le plus à gauche et le plus à droite sont sélectionnés pour tracer respectivement les lignes de voie gauche et droite. La courbe ajustée est finalement superposée en couleurs sur l'image BEV et enregistrée comme résultat final [4].

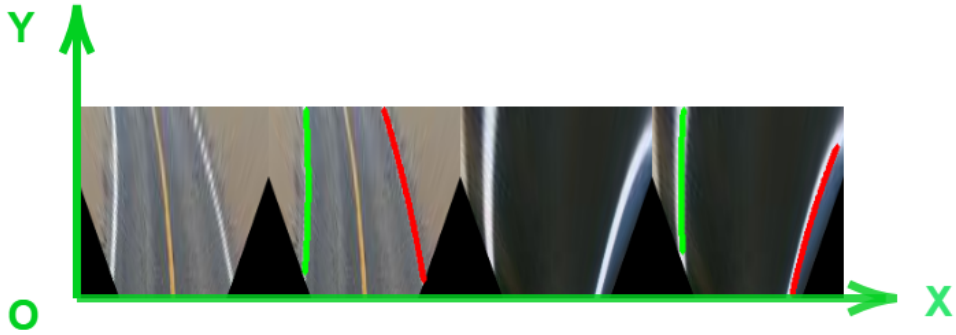


FIGURE 7 – Image BEV et courbes ajustées représentant les lignes de voie.

3.1.8 Calcul des coordonnées de la ligne de voie, de l'angle de déviation du véhicule et du décalage latéral

Après l'ajustement par un polynôme du second degré, le modèle de voie est représenté par une courbe continue de la forme $x = ay^2 + by + c$. Cependant, pour l'affichage dans l'image, la sortie JSON et l'utilisation dans les modules de commande, cette courbe doit être discrétisée en points de coordonnées en pixels [4].

Dans cette expérimentation, nous appliquons un échantillonnage uniforme sur la hauteur utile de l'image. Nous prélevons plusieurs valeurs de y, puis calculons les valeurs correspondantes de x à partir de la fonction ajustée afin de générer une série de points ordonnés.

Les points obtenus sont triés de haut en bas, formant un nuage de points structuré pour la ligne de voie. Si une seule ligne est détectée, seul son ensemble de points est exporté. Si deux lignes ou plus sont détectées, des ensembles distincts sont générés pour la voie gauche et pour la voie droite. À partir de ces ensembles, nous calculons l'angle de déviation par rapport à l'axe vertical, ainsi que la courbure et le décalage du centre du véhicule.

Les résultats finaux sont enregistrés au format JSON pour être utilisés dans le système de contrôle du véhicule. En parallèle, les distances du véhicule aux deux lignes, la largeur estimée de la voie et le décalage du centre sont conservés en pixels afin de faciliter une calibration métrique ultérieure.

Résumé. L'ensemble du processus commence par l'image frontale en entrée, suivie de la correction de perspective, du filtrage colorimétrique et spatial, de l'optimisation morphologique, puis de l'extraction des caractéristiques et de l'analyse par clustering, pour finalement obtenir deux lignes de voie lissées. Cette conception modulaire garantit l'indépendance et la flexibilité de chaque étape, tout en apportant une bonne robustesse sur des scènes de test variées, conformément aux tendances observées dans les approches classiques de détection de voie [2, 4].

3.1.9 Principales difficultés du module de perception

1. **Sensibilité aux conditions d'éclairage et à l'environnement**

   Dans des environnements complexes (ombres, fortes luminosités, réflexions), la détection des lignes de voie basée sur la couleur devient instable et sujette à des erreurs, ce qui est une difficulté classique largement signalée dans la littérature [2, 5].

   Nous utilisons l'espace colorimétrique HSV, plus robuste aux variations de luminosité, combiné avec une sélection de région d'intérêt (ROI) et un filtrage en plusieurs étapes

afin de réduire les interférences et d'améliorer la stabilité de la détection [5, 2].

2. **Forte dépendance aux paramètres**

   Plusieurs étapes de l'algorithme reposent sur des paramètres définis manuellement (seuils, ROI, etc.), ce qui limite la capacité de généralisation et nécessite des ajustements fréquents selon les différentes scènes, comme le soulignent les travaux de synthèse sur la détection de voie [2].

   Nous avons adopté une architecture modulaire, permettant d'analyser et d'ajuster chaque étape indépendamment, ce qui améliore la lisibilité, la flexibilité et le contrôle du système.

3. **Compromis entre complexité algorithmique et temps réel**

   Le système doit fonctionner en temps réel, alors que des méthodes plus complexes augmentent fortement le coût de calcul et deviennent difficiles à intégrer sur une plateforme embarquée, ce qui constitue un compromis récurrent dans les systèmes de lane keeping embarqués [2, 3].

   Nous avons privilégié des méthodes classiques de vision par ordinateur, légères et efficaces (BEV, squelettisation, clustering DBSCAN), ainsi qu'une optimisation de la transmission des données pour garantir une exécution rapide.

3.2 Commande

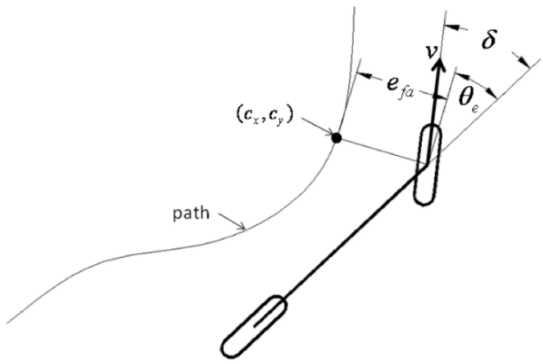
L'un des objectifs principaux est de développer un système de contrôle capable de suivre la trajectoire donnée de manière autonome. Le contrôleur Stanley a été choisi pour cette tâche en raison de sa capacité à corriger efficacement à la fois l'écart latéral du véhicule par rapport à la trajectoire et la différence d'orientation entre le véhicule et la voie [1].

Dans le pipeline global, le contrôleur reçoit les informations issues du module de perception, notamment le décalage latéral (*offset*) et l'angle de la trajectoire (*path angle*). À partir de ces données, il génère les commandes de direction et de vitesse (*throttle*) nécessaires pour assurer un suivi de trajectoire stable, conformément au principe du

contrôleur Stanley [1].

3.2.1 Principe du contrôleur Stanley

Le contrôleur Stanley repose sur la correction de deux erreurs principales : l'écart latéral (cross-track error, CTE) et l'écart angulaire (heading error). L'écart latéral correspond à la distance entre le véhicule et la trajectoire, tandis que l'écart angulaire représente la différence entre l'orientation du véhicule et celle de la trajectoire, conformément aux méthodes décrites par Snider [1].



La commande de direction est donnée par :

$$\delta = \theta_e + \arctan\left(\frac{k \cdot e}{v}\right)$$

où δ est l'angle de braquage, θ_e l'écart angulaire, e l'écart latéral (filtré), v la vitesse du véhicule et k un gain ajustable.

Dans l'implémentation, une vitesse minimale est utilisée dans ce calcul afin d'éviter des instabilités lorsque le véhicule roule très lentement, ce qui correspond à une adaptation pratique classique du contrôleur Stanley [1].

3.2.2 Architecture générale

Le système de contrôle s'inscrit dans une chaîne perception–commande. Le module perception fournit l'offset latéral et l'angle de la trajectoire. Ces informations sont ensuite

utilisées par le contrôleur Stanley pour calculer l'angle de braquage ainsi qu'une commande de vitesse. Les commandes sont finalement envoyées à l'API DonkeyCar pour piloter le véhicule [1].

3.2.3 Fonctionnement du code

Le code de contrôle implémente plusieurs étapes successives. Tout d'abord, un biais peut être appliqué sur l'offset afin de corriger un éventuel décalage systématique du capteur. Ensuite, l'erreur latérale est filtrée à l'aide d'un filtre exponentiel (EMA) pour limiter les variations brusques.

L'erreur d'orientation est calculée à partir de l'angle de la trajectoire et de l'orientation actuelle du véhicule. La commande Stanley est alors déterminée en combinant cette erreur avec le terme correctif dépendant de l'écart latéral et de la vitesse [1].

Une saturation est appliquée à l'angle de braquage afin de respecter les limites physiques du véhicule ($\pm 30°$). De plus, une limitation de la vitesse de variation de cet angle (*rate limiting*) est implémentée en fonction du temps entre deux itérations, afin d'éviter des changements trop brusques.

La commande de direction est ensuite normalisée entre -1 et 1 pour être compatible avec l'API DonkeyCar.

Concernant la vitesse, une commande simple est utilisée : une valeur de base est définie et ajustée avec des bornes minimale et maximale. Un mécanisme permet également d'éviter le calage du véhicule lorsque la vitesse est trop faible.

3.2.4 Paramètres principaux

Les principaux paramètres du contrôleur sont les suivants :

- Gain Stanley : $k = 1.5$, permettant de régler la réactivité du suivi de trajectoire
- Filtrage EMA : $\alpha = 0.3$, utilisé pour lisser l'erreur latérale
- Angle maximal de braquage : $\pm 30°$
- Vitesse minimale pour le calcul : évite les instabilités à basse vitesse

- Seuil de gel (*freeze*) : permet d'annuler la direction lorsque le véhicule est quasi immobile
- Paramètres de throttle : valeur de base et bornes min/max pour garantir un mouvement stable

3.2.5 Points d'attention

Contrairement à une implémentation purement théorique, plusieurs adaptations ont été nécessaires pour assurer un comportement réaliste. En particulier, la gestion des basses vitesses, le filtrage du bruit et la limitation des variations de commande jouent un rôle essentiel dans la stabilité globale du véhicule.

Par ailleurs, la fréquence d'exécution du contrôleur dépend directement de la boucle principale (via le temps réel), et n'est pas fixée explicitement à 100 Hz dans le code. Le *rate limiting* s'adapte dynamiquement à ce pas de temps.

3.2.6 Simulation

Avant toute intégration sur le DonkeyCar réel, une simulation logicielle du contrôleur Stanley a été développée afin d'analyser son comportement de manière isolée. Cette simulation permet de vérifier la cohérence de l'algorithme de contrôle indépendamment du module de perception et du matériel embarqué.

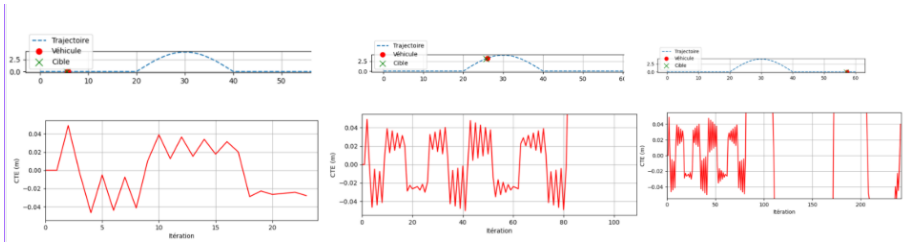


FIGURE 8 – Résultats de la simulation logicielle

Une animation est générée afin de visualiser en temps quasi réel l'évolution du véhicule par rapport à la trajectoire de référence.Cette animation permet d'observer la convergence du véhicule vers la trajectoire, ainsi que son comportement lors du passage en courbe. En

parallèle, l'évolution de l'orientation du véhicule (yaw) et de l'erreur latérale est tracée, ce qui facilite l'analyse de la stabilité du contrôleur.

3.3 Calibration et mesures

La calibration de la caméra a été réalisée à l'aide d'un échiquier standard, permettant d'obtenir les paramètres intrinsèques (matrice caméra) et extrinsèques (coefficients de distorsion) stockés dans le fichier `camera_calibration.pkl`. Cette correction d'undistorsion est appliquée à chaque image via la classe `CameraUndistorter` avant la détection de voie, réduisant les erreurs de perspective dues à la lentille.

L'odométrie est estimée à partir d'un encodeur sur une roue (pin GPIO 13, 20 impulsions par tour, rayon 23 mm), avec un filtre EMA ($\alpha = 0.3$) pour lisser le bruit. La vitesse instantanée et le yaw sont ainsi calculés et fournis au contrôleur Stanley. Les signaux de perception (`offset_m`, `angle_deg`) et de commande (`steer`, `throttle`, `current_speed`) sont enregistrés en temps réel dans des fichiers CSV via `DataLoggerPart` (dossier `data_plot`).

L'analyse hors-ligne est assurée par le script `generate_plots.py`, qui charge le dernier CSV et génère des courbes de perception et de commande (offset/angle en haut, steer/-throttle en bas), permettant de valider la cohérence entre les sorties des sous-systèmes et d'ajuster les gains du contrôleur.

4 Résultats et Discussion

FIGURE 9 – Exemple de fonctionnement : perception (ligne projetée, offset = -0.14 m) et commande (steer = 0.14, throttle = 0.30).

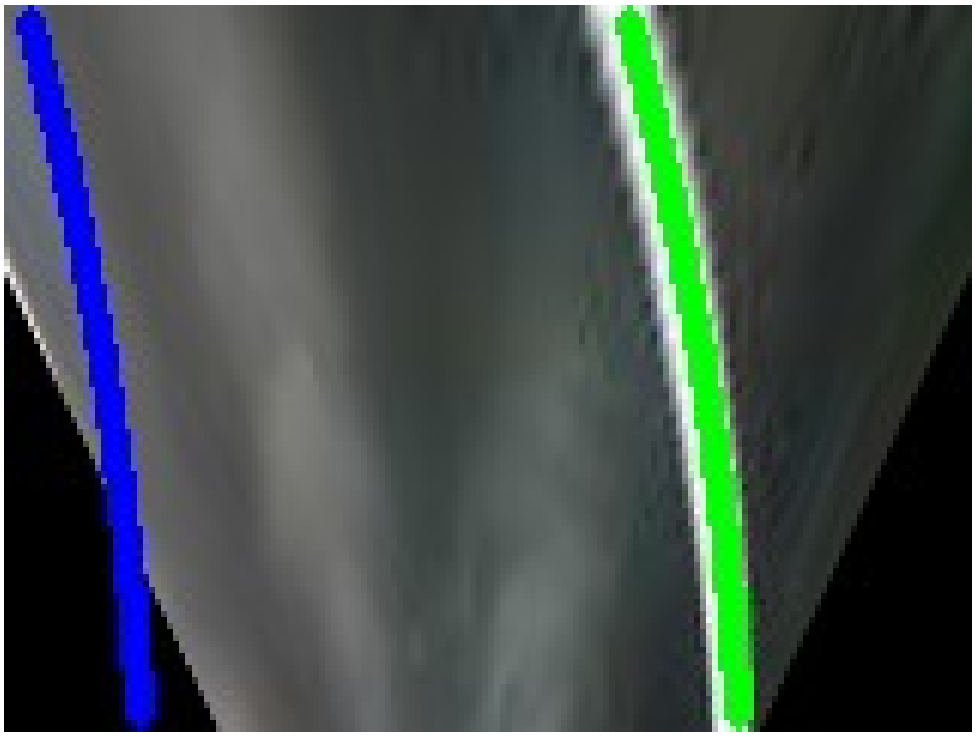


FIGURE 10 – Détection de voie : ligne détectée (droite) et projection parallèle (gauche).

Les essais ont été réalisés sur une piste rectiligne et courbe, avec des vitesses comprises entre 0.3 et 0.5 m/s. Le robot suit correctement la voie dans les conditions nominales, avec une erreur latérale moyenne de 2 à 5 cm et une stabilité satisfaisante grâce au contrôleur Stanley. Les courbes générées par `generate_plots.py` montrent une corrélation claire entre l'offset de perception et les consignes de braquage, validant l'intégration des sous-systèmes.

Les principales limites identifiées sont la sensibilité aux ombres et aux variations d'éclairage pour la détection de voie, ainsi que la précision de l'odométrie à basse vitesse. Des pistes d'amélioration incluent un filtre adaptatif pour l'odométrie, une calibration dynamique de la caméra et l'ajout d'un IMU pour une meilleure estimation du yaw.

5 Conclusion

Le projet a atteint son objectif principal : un DonkeyCar autonome utilisant une perception visuelle et un contrôle classique, avec une architecture modulaire basée sur les

parts de DonkeyCar. Les outils de calibration, de mesure et d'analyse hors ligne développés facilitent le réglage et la validation. Des améliorations sur la robustesse à l'éclairage et sur la précision de l'odométrie permettraient une performance accrue sur des pistes plus complexes.

6 Bibliographie

[1] Jarrod M. Snider. *Automatic Steering Methods for Autonomous Automobile Path Tracking*. Rapp. tech. CMU-RI-TR-09-08. Pittsburgh, PA, USA : Robotics Institute, Carnegie Mellon University, fév. 2009.

[2] S. P. Narote et al. « A review of recent advances in lane detection and departure warning system ». In : *Pattern Recognition* 73 (2018), p. 216-234. doi : 10.1016/j.patcog.2017.08.014.

[3] C. Y. Kuo, Y. R. Lu et S. M. Yang. « On the image sensor processing for lane detection and control in vehicle lane keeping systems ». In : *Sensors* 19.7 (2019), p. 1665. doi : 10.3390/s19071665.

[4] Jiaxiang Niu et al. « Robust lane detection using two-stage feature extraction with curve fitting ». In : *Pattern Recognition* 59 (2016), p. 225-233. doi : 10.1016/j.patcog.2015.12.010.

[5] K.-Y. Chiu et S.-F. Lin. « Lane detection using color-based segmentation ». In : *Proceedings of the 2005 IEEE Intelligent Vehicles Symposium*. Las Vegas, NV, USA, 2005, p. 706-711. doi : 10.1109/IVS.2005.1505186.

[6] John Canny. « A computational approach to edge detection ». In : *IEEE Transactions on Pattern Analysis and Machine Intelligence* PAMI-8.6 (1986), p. 679-698. doi : 10.1109/TPAMI.1986.4767851.

[7] T. Y. Zhang et C. Y. Suen. « A fast parallel algorithm for thinning digital patterns ». In : *Communications of the ACM* 27.3 (1984), p. 236-239. doi : 10.1145/357994.358023.

[8] Martin Ester et al. « A density-based algorithm for discovering clusters in large spatial databases with noise ». In : *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*. Portland, OR, USA, 1996, p. 226-231. url : https://cdn.aaai.org/KDD/1996/KDD96-037.pdf.